100 XP

Understand integration with VS Code

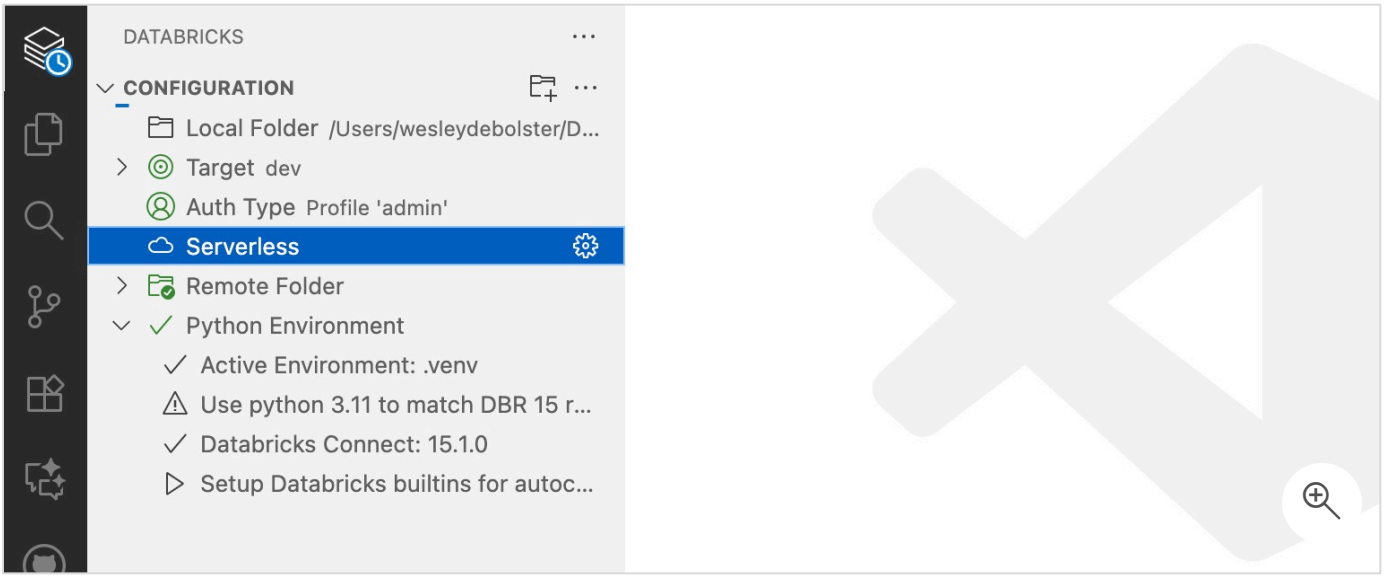
7 minutes

Data engineers working with Azure Databricks often switch between their local development environment and remote workspaces. You write code locally in your preferred editor, then copy it to Databricks notebooks or upload files to test on clusters. This back-and-forth workflow slows down development and makes debugging difficult. The Databricks extension for Visual Studio Code changes this by connecting your local development environment directly to your remote Azure Databricks workspace.

With this integration, you develop in Visual Studio Code using familiar tools and shortcuts while executing code on Azure Databricks compute resources. You don't need to leave your editor to run notebooks, test Python scripts, or deploy workflows. This approach keeps your development environment consistent while taking advantage of Databricks' distributed computing power.

Develop locally and execute remotely

Visual Studio Code provides a development environment on your local machine with features like **IntelliSense**, **syntax highlighting**, and **Git integration**. The Databricks extension adds the ability to execute your local code on remote Azure Databricks clusters or serverless compute. You write code in `.py` files or notebooks on your computer, then run them directly on Azure Databricks infrastructure without manually copying files or switching between applications.



You can also run Python, R, Scala, and SQL notebooks as **Lakeflow Jobs**. Instead of executing code interactively on a cluster, you package your notebook as a job that runs on a schedule or trigger. This capability bridges local development and production deployment, letting you test job configurations before committing them to source control.

The extension supports multiple Databricks projects within a single Visual Studio Code workspace. If you work on different Databricks environments or modules, you switch between project configurations without closing and reopening folders. Each project maintains its own workspace connection, authentication settings, and cluster selection.

Debug code with Databricks Connect

The Databricks extension integrates **Databricks Connect** to enable full debugging capabilities within Visual Studio Code.

Databricks Connect creates a direct connection between your local Python environment and a remote Azure Databricks cluster. When you debug code, the Python debugger in Visual Studio Code controls execution on the cluster. You set breakpoints, inspect variables, and step through code just as you would with local Python scripts. The difference is that your code runs on Databricks compute with access to distributed data and Spark APIs.

This debugging environment works with both Python files and notebooks. You can debug notebooks cell by cell, examining the state of DataFrames and variables at each step. When you encounter an error, you modify your code locally, set a breakpoint before the problematic line,

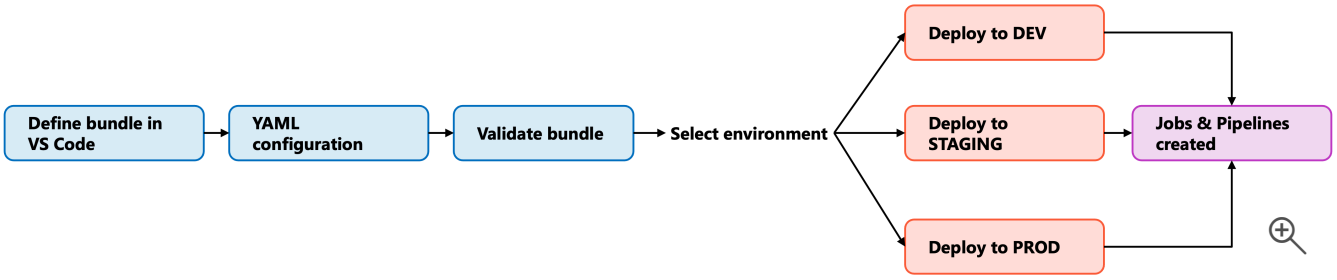
and rerun the debugger to investigate. This tight feedback loop reduces the time spent troubleshooting data pipelines and transformations.

Beyond interactive debugging, the extension supports running Python tests with `pytest`. You write unit tests for your data processing logic and execute them on Databricks compute. This capability ensures that your tests run against the same environment and dependencies as your production code, catching environment-specific issues early in development.

Deploy workflows with Declarative Automation Bundles

The Databricks extension simplifies deployment through **Declarative Automation Bundles** (formerly known as Databricks Asset Bundles), which package your code, configurations, and dependencies into deployable units.

Declarative Automation Bundles define **Lakeflow Jobs**, **Lakeflow Spark Declarative Pipelines (SDP)**, and **MLOps Stacks** using configuration files. You specify compute settings, schedule triggers, and task dependencies in YAML format. The extension provides a UI for creating, validating, and deploying these bundles without leaving Visual Studio Code.



When you're ready to deploy, the extension applies your bundle to a target environment (development, staging, or production). It creates or updates jobs, configures clusters, and sets up permissions according to your specifications. This declarative approach ensures consistent deployments and makes it easier to apply CI/CD patterns to your data engineering workflows.

With bundles, you can version your entire workflow configuration alongside your code in Git. Changes to job schedules, cluster sizes, or task dependencies go through the same review and approval process as code changes. This practice improves collaboration and provides an audit trail for production deployments.

Synchronize code between local and remote workspaces

While Asset Bundles handle deployment, you might also want to keep local code synchronized with workspace folders for quick prototyping or collaboration. The extension supports one-way automatic synchronization from your local Visual Studio Code project to workspace directories. You edit files locally, and changes automatically upload to the workspace. The workspace files are intended to be transient, so you should avoid making changes directly in the workspace, as these changes won't sync back to your local project.

This synchronization capability works well for teams transitioning from workspace-based development to local development. You maintain existing workspace folders while gradually adopting local development practices and version control. By syncing your local files to the workspace, you can test and run them remotely while keeping your local project as the source of truth for version control in Git.

However, for production workflows, Declarative Automation Bundles provide better control and consistency. Synchronization suits exploratory work and quick iterations, while bundles suit structured deployment pipelines with environment-specific configurations.

Next unit: Understand integration with Power Platform

Previous Next